

دانشگاه تهران  
پردیس دانشکده‌های فنی  
دانشکده مهندسی برق و کامپیوتر



# ابزار دقیق

## تمرین سری پنجم

نام و نام خانوادگی:

امیرعلی دهقانی

شماره دانشجویی:

۸۱۰۱۰۲۴۴۳

## فهرست مطالب

سوال ۱

۲

سوال ۲

۶

سوال ۳

۱۱

سوال ۴

۱۴

## سوال ۱

## الف) راه اندازی موتور DC و ضرورت استفاده از درایور

برای کنترل یک موتور DC توسط میکروکنترلر، هرگز نباید موتور را مستقیماً به پایه‌های پردازنده متصل کرد. به این دلیل که:

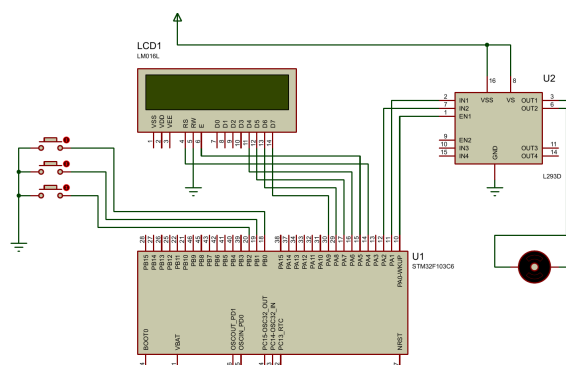
- **محدودیت جریان دهی:** پایه‌های ورودی و خروجی در میکروکنترلرها، تنها قادر به تامین جریان‌های بسیار ضعیف (در حد چند میلی‌آمپر) هستند. در حالی که یک موتور DC حتی در حالت بدون بار، به ده‌ها یا صدها میلی‌آمپر جریان برای راه‌اندازی نیاز دارد. کشیدن این جریان بالا به صورت مستقیم، باعث سوختن پایه و حتی آسیب دیدن کامل پردازنده می‌شود.
- **تفاوت سطح ولتاژ:** میکروکنترلری مانند STM32 با ولتاژ ۳.۳ ولت کار می‌کند، در حالی که موتورهای DC معمولاً برای کار با ولتاژهای بالاتری مثل ۵، ۱۲ یا ۲۴ ولت طراحی شده‌اند.
- **ولتاژ برگشتی:** موتورهای دارای سیم‌پیچ هستند. وقتی موتور متوقف می‌شود یا جهت چرخش آن تغییر می‌کند، یک ولتاژ القایی بزرگ و معکوس تولید می‌شود. اگر موتور مستقیماً به میکروکنترلر وصل باشد، این ولتاژ معکوس وارد مدار منطقی شده و پردازنده را از بین می‌برد.

## نقش درایور موتور:

درایور موتور به عنوان یک واسطه بین میکروکنترلر و موتور عمل می‌کند. این قطعه، سیگنال‌های کنترلی را از میکروکنترلر دریافت کرده و با استفاده از مدار داخلی خود، ولتاژ و جریان اصلی منبع تغذیه مجزا را به موتور اعمال می‌کند. همچنین این درایورها دارای دیودهای هرزگرد (محافظ) داخلی هستند که از ورود ولتاژهای مخرب به میکروکنترلر جلوگیری می‌کنند.

## ب) طراحی مدار در نرم‌افزار پروتئوس

در این بخش، مدار مورد نیاز شامل میکروکنترلر STM32، درایور موتور L293D، موتور DC، نمایشگر LCD و کلیدهای کنترلی در نرم‌افزار پروتئوس طراحی شد. پایه‌ها به گونه‌ای متصل شده‌اند که بتوان با استفاده از سیگنال PWM سرعت موتور را کنترل کرد و با کمک پایه‌های دیجیتال، جهت چرخش آن را تغییر داد. نمای کلی اتصالات در شکل زیر قابل مشاهده است.

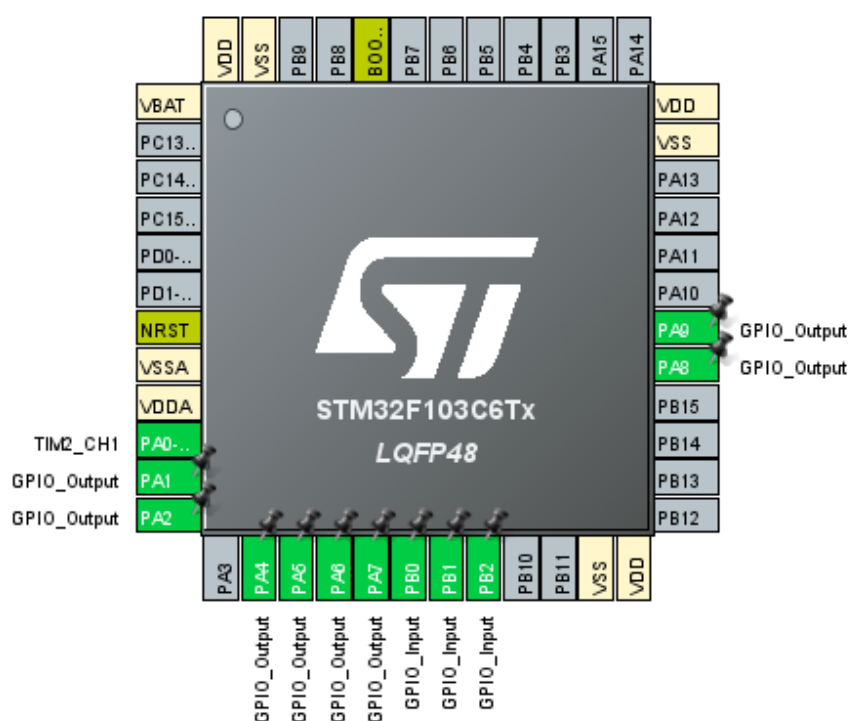


شکل ۱: شماتیک مدار کنترل موتور DC در نرم‌افزار پروتئوس

## ج) انتخاب پایه‌ها و تنظیمات میکروکنترلر

برای راه‌اندازی سیستم، تنظیمات پایه‌های میکروکنترلر STM32F103C6 در نرم‌افزار STM32CubeIDE به این شکل انجام شد:

- **تنظیمات PWM (کنترل سرعت):** برای تولید سیگنال PWM، از تایمر ۲ استفاده شد. کانال ۱ از این تایمر فعال شد که به صورت خودکار پایه PA0 را به عنوان خروجی PWM تنظیم می‌کند.
- **تنظیمات خروجی (کنترل جهت و نمایشگر):** پایه‌های PA1 و PA2 به عنوان خروجی برای اتصال به درایور موتور در نظر گرفته شدند. همچنین پایه‌های PA4 تا PA9 نیز به عنوان خروجی دیجیتال برای ارسال اطلاعات به نمایشگر LCD تنظیم شدند.
- **تنظیمات ورودی (کلیدهای کنترلی):** پایه‌های PB0، PB1 و PB2 برای اتصال کلیدهای فشاری به عنوان ورودی انتخاب شدند. برای جلوگیری از نویز و ایجاد سطح منطقی پایدار، مقاومت‌های Pull-up داخلی میکروکنترلر برای این پایه‌ها فعال شدند.

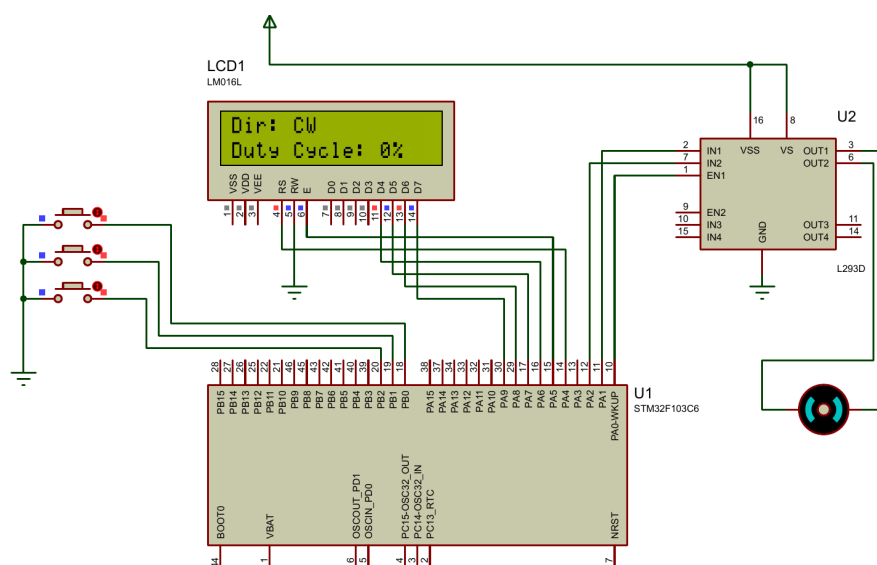


شکل ۲: تنظیمات پایه‌های میکروکنترلر در نرم‌افزار STM32CubeIDE

## (د) پیاده‌سازی وضعیت اولیه سیستم

در این بخش، برنامه میکروکنترلر طوری نوشته شد که سیستم در لحظه شروع به کار، کاملاً پایدار باشد. به همین دلیل، مقدار دیوتی ساینک PWM را روی صفر تنظیم کردیم تا موتور هیچ حرکتی نداشته باشد. وضعیت پایه‌های درایور موتور هم در حالت پیش‌فرض روی چرخش ساعت‌گرد (CW) تنظیم شد. بعد از روشن شدن نمایشگر LCD، همین وضعیت اولیه یعنی جهت چرخش و مقدار دیوتی ساینک روی صفحه چاپ می‌شود.

همان‌طور که در تصویر شبیه‌سازی مشخص است، در لحظه شروع، موتور ثابت مانده و اطلاعات به درستی روی نمایشگر نشان داده شده‌اند.

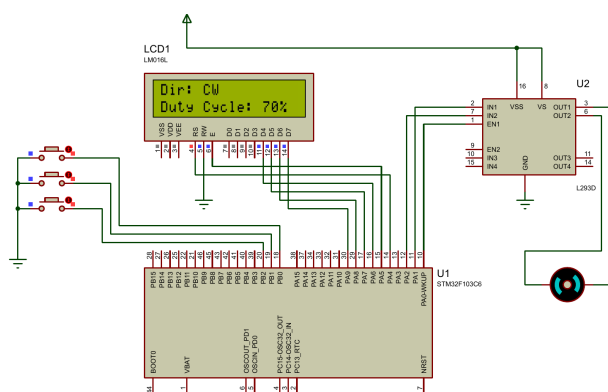


شکل ۳: خروجی شبیه‌سازی در حالت اولیه (موتور متوقف و دیوتی ساینک صفر)

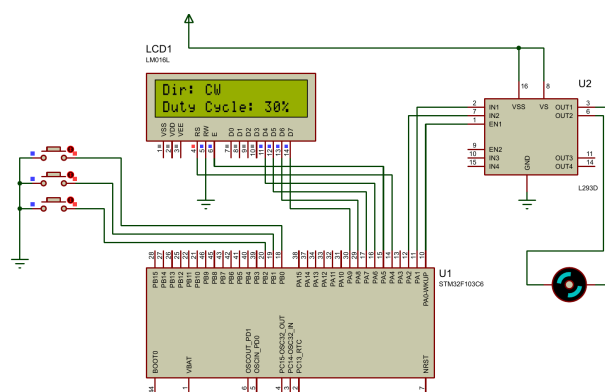
## (ه) کنترل سرعت موتور با استفاده از کلیدهای فشاری

در این قسمت، قابلیت تغییر سرعت موتور به برنامه اضافه شد. منطق کد به این شکل است که با فشردن کلید اول، مقدار دیوتی ساینک PWM به اندازه ۱۰ درصد بیشتر می‌شود و در نتیجه موتور سریع‌تر می‌چرخد. با زدن کلید دوم هم این مقدار ۱۰ درصد کاهش پیدا می‌کند. برای جلوگیری از بروز خطا در عملکرد سیستم، شرطهایی در برنامه قرار داده شده است تا دیوتی ساینک هیچ‌وقت از صفر کمتر و از ۱۰۰ درصد بیشتر نشود. با هر بار تغییر سرعت، درصد جدید بلافاصله روی نمایشگر به‌روز می‌شود.

در تصاویر زیر عملکرد مدار بعد از چند بار فشردن کلیدهای افزایش و کاهش سرعت مشاهده می‌شود که دیوتی سایکل به درستی روی مقادیر ۳۰ و ۷۰ درصد تنظیم شده است.



(ب) تنظیم دیوتی سایکل روی ۷۰ درصد

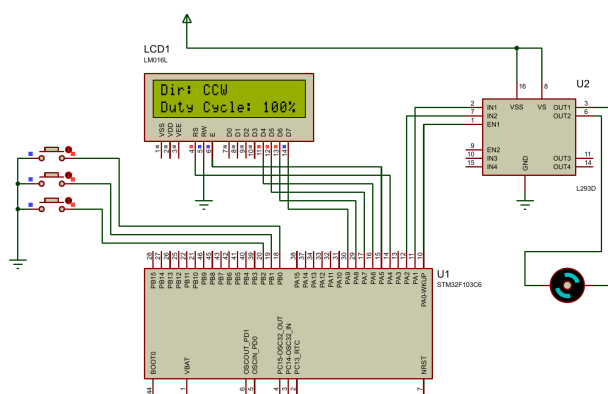


(T) تنظیم دیوتی سایکل روی ۳۰ درصد

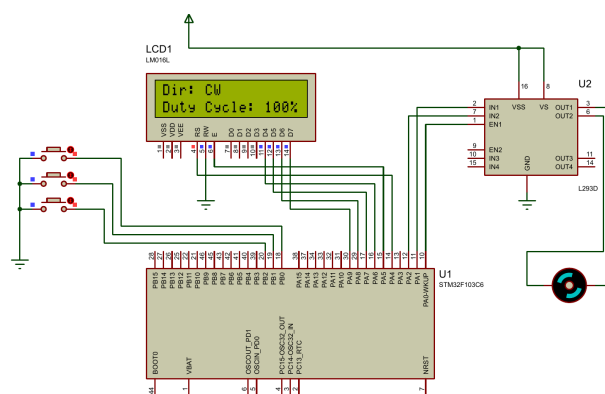
شکل ۴: خروجی مدار برای کنترل سرعت با کلیدهای فشاری

## (و) تغییر جهت چرخش موتور

در این بخش، قابلیت تغییر جهت چرخش موتور به برنامه اضافه شد. منطق کار به این صورت است که با فشردن کلید سوم، وضعیت پایه‌های خروجی متصل به درایور موتور معکوس می‌شود. در نتیجه، جهت چرخش موتور از حالت ساعت‌گرد (CW) به پادساعت‌گرد (CCW) تغییر پیدا می‌کند. این تغییر وضعیت به صورت لحظه‌ای روی نمایشگر LCD نیز آپدیت می‌شود. در تصاویر زیر خروجی مدار پیش از فشردن کلید سوم و پس از آن در دیوتی سایکل ۱۰۰ درصد مشاهده می‌شود. همچنین فیلم اجرای کامل این بخش در پوشه Q1 قرار داده شده است.



(ب) چرخش موتور در جهت پادساعت‌گرد (CCW)



(T) چرخش موتور در جهت ساعت‌گرد (CW)

شکل ۵: خروجی مدار برای تغییر جهت چرخش موتور

## سوال ۲

## الف) بررسی مزایای استپر موتور در سیستم ردیاب خورشیدی

در ابتدا مزایای استپر موتور نسبت به موتور DC معمولی برای استفاده در سیستم‌های ردیاب خورشیدی بررسی می‌شود. پنل‌های خورشیدی برای دریافت بیشترین توان، باید مسیر حرکت خورشید را با دقت بالا دنبال کنند. استپر موتورها به دلیل ویژگی‌های ساختاری خود، برای این کاربرد بسیار مناسب‌تر هستند که در ادامه به سه مورد از مهم‌ترین آن‌ها اشاره شده است:

- **دقت موقعیتیابی:** برخلاف موتورهای DC که به صورت پیوسته می‌چرخند، استپر موتورها با دریافت هر پالس الکتریکی، به اندازه یک زاویه مشخص و ثابت حرکت می‌کنند. این ویژگی باعث می‌شود که بتوان زاویه پنل را با دقت بسیار بالایی تنظیم کرد.

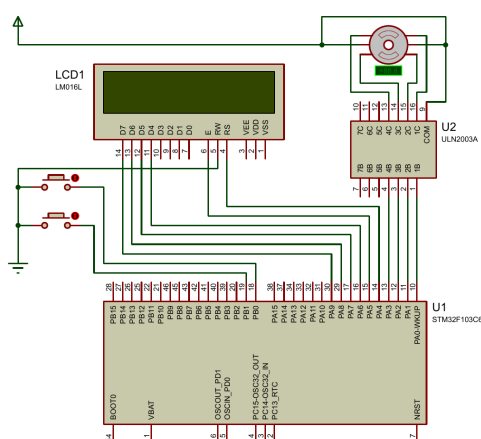
- **کنترل حلقه باز:** در موتورهای DC برای اطلاع از موقعیت شفت موتور، به سنسورهای فیدبک مانند انکودر نیاز است که طراحی سیستم را پیچیده می‌کند (سیستم حلقه بسته). اما در استپر موتورها، پردازنده با شمارش تعداد پالس‌های ارسال شده، به طور دقیق می‌داند موتور در چه زاویه‌ای قرار دارد و نیازی به سنسور فیدبک نیست و به همین دلیل می‌توانیم از کنترل حلقه باز استفاده بکنیم.

- **گشتاور نگهدارنده (Holding Torque):** پنل‌های خورشیدی در فضای باز نصب می‌شوند و همواره تحت تاثیر نیروهای خارجی مانند وزش باد قرار دارند. استپر موتورها در حالتی که متوقف هستند اما جریان برق در سیم‌پیچ‌های آن‌ها برقرار است، گشتاور بالایی برای حفظ موقعیت خود تولید می‌کنند. این گشتاور نگهدارنده مانع از تغییر زاویه پنل در اثر عوامل محیطی می‌شود، در حالی که موتورهای DC برای ثابت ماندن در یک زاویه نیازمند ترمز مکانیکی یا کنترلرهای پیچیده هستند.

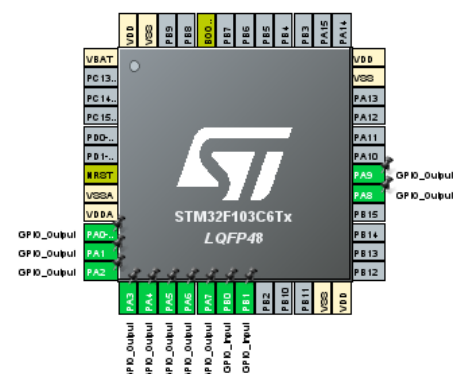
## تنظیمات اولیه

قبل از ورود به بخش شبیه‌سازی، شماتیک سخت‌افزاری و تنظیمات اولیه میکروکنترلر آماده شد. در این طراحی، از درایور ULN2003 برای راه‌اندازی و اتصال میکروکنترلر به استپر موتور استفاده شده است. نمایشگر LCD و کلیدهای فشاری نیز برای نمایش اطلاعات و دریافت فرمان‌های کاربر به مدار اضافه شدند. تنظیمات پایه‌ها در نرم‌افزار STM32CubeIDE به گونه‌ای انجام شده که پایه‌های متصل به درایور و نمایشگر به عنوان خروجی، و پایه‌های متصل به کلیدها به عنوان ورودی (با مقاومت Pull-up داخلی) تنظیم شوند.

در تصاویر زیر، شماتیک طراحی شده در محیط پروتئوس و همچنین وضعیت تنظیمات پایه‌ها در نرم‌افزار قابل مشاهده است.



(ب) شماتیک مدار راه‌انداز استپر موتور در پروتئوس



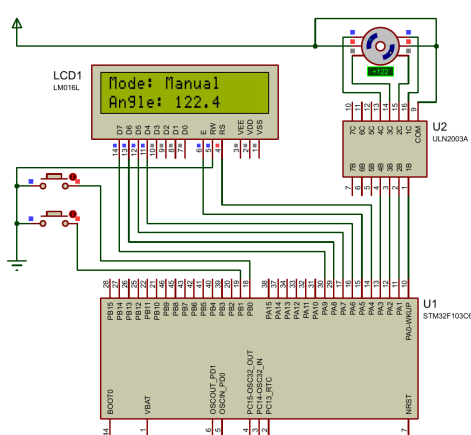
(آ) تنظیمات پایه‌ها در نرم‌افزار STM32CubeIDE

شکل ۶: تنظیمات اولیه نرم‌افزاری و سخت‌افزاری برای سیستم ریدای خورشیدی

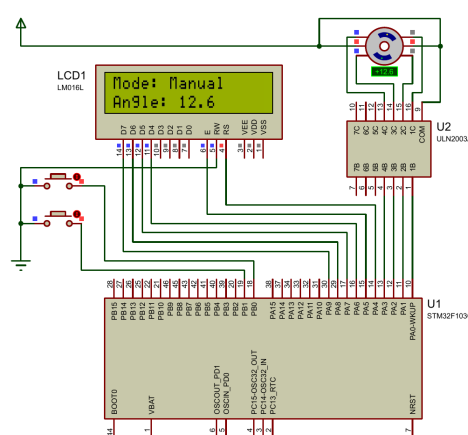
## (ب) کنترل دستی موقعیت استپر موتور

در این بخش، قابلیت کنترل موقعیت استپر موتور به صورت دستی پیاده‌سازی شد. از آنجا که زاویه گام (Step Angle) این موتور برابر با ۸.۱ درجه است، با فشردن کلید اول، موتور دقیقاً به اندازه ۸.۱ درجه در جهت ساعت‌گرد چرخیده و یک پله به جلو می‌رود. به همین ترتیب با فشردن کلید دوم، زاویه موتور به اندازه ۸.۱ درجه در جهت پادساعت‌گرد کاهش می‌یابد (با قابلیت چرخش در زوایای منفی). میکروکنترلر در هر لحظه تعداد پله‌های طی شده را شمارش کرده و با ضرب آن در زاویه هر گام، زاویه فعلی پل را خورشیدی را محاسبه و روی نمایشگر LCD چاپ می‌کند.

همچنین برای رفع مشکل پرش اولیه روتور در شبیه‌ساز پروتئوس، در لحظه راه‌اندازی مدار، فازهای مربوط به موقعیت صفر مطلق فعال شدند تا موتور پیش از دریافت اولین فرمان در زاویه صفر درجه قفل شود و تغییرات زاویه کاملاً خطی و بدون خطا صورت گیرد. در تصاویر زیر خروجی عملکرد مدار در حالت دستی و در زوایای مختلف قابل مشاهده است.



(ب) تنظیم دستی موقعیت روی زاویه ۴.۱۲۲ درجه



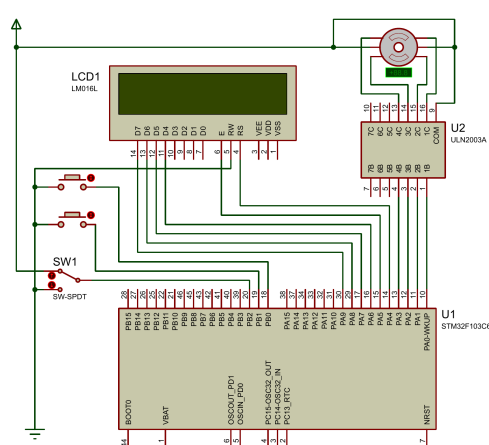
(آ) تنظیم دستی موقعیت روی زاویه ۶.۱۲ درجه

شکل ۷: خروجی مدار در حالت کنترل دستی و نمایش زاویه روی نمایشگر

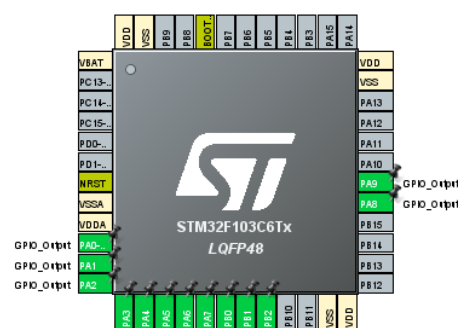
## پ) پیاده‌سازی سیستم کنترل خودکار با توقف در ۱۸۰ درجه

در این بخش، قابلیت کنترل خودکار به سیستم ردياب خورشیدی اضافه شد. برای تغییر بین دو حالت دستی و خودکار، از یک کلید استفاده شده که به پایه PB2 میکروکنترلر متصل است. با تغییر وضعیت این کلید، سیستم وارد حالت Auto می‌شود. منطق برنامه‌نویسی به گونه‌ای است که در حالت خودکار، میکروکنترلر هر دو ثانیه یک‌بار فرمان حرکت به اندازه یک پله ( $1/8^\circ$ ) در جهت ساعت‌گرد را به درایور ارسال می‌کند. سیستم با شمارش دقیق پالس‌ها، موقعیت را ردیابی کرده و این روند تا رسیدن پنل به زاویه  $180^\circ$  ادامه می‌یابد و پس از آن متوقف می‌شود.

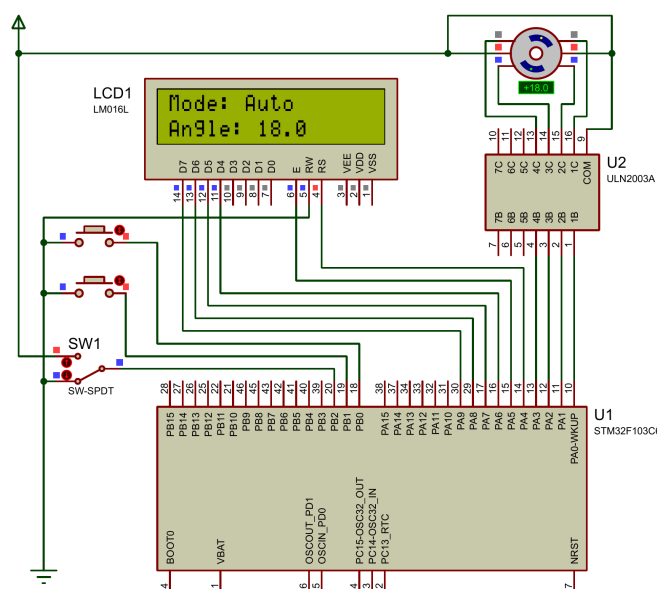
در صورتی که مدار در زاویه‌ای دلخواه از حالت دستی به حالت خودکار تغییر وضعیت دهد، حرکت از همان زاویه ادامه پیدا می‌کند و سیستم دچار خطا نمی‌شود. در تصاویر زیر، تنظیمات جدید پیکرینت، شماتیک جدید، و تصویری از اجرای سیستم در حالت خودکار آورده شده است.



ب) شماتیک مدار با اضافه شدن کلید SW-SPDT



ت) تنظیم پایه PB2 به عنوان ورودی در STM32CubeIDE



ج) خروجی عملکرد مدار در حالت خودکار (Auto)

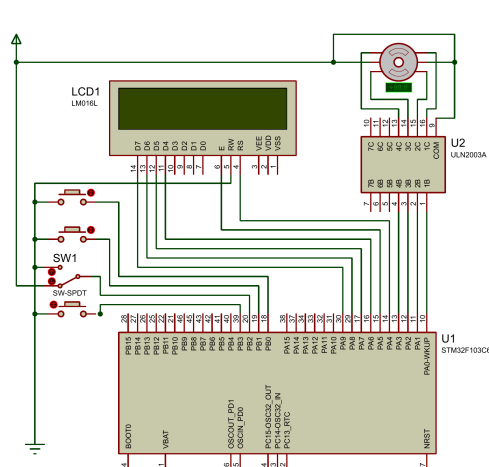
شکل ۸: پیاده‌سازی و تست کنترل خودکار ردیاب خورشیدی

## (ت) پیاده‌سازی کلید بازگشت به موقعیت اولیه (Reset)

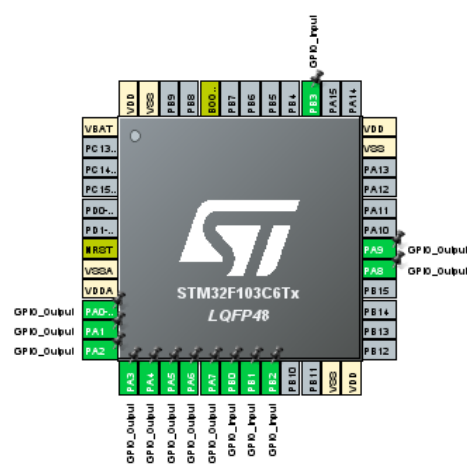
در این بخش، کلید بازگشت (Reset) برای برگرداندن پِنل خورشیدی به موقعیت اولیه به مدار اضافه شد. این کلید به صورت Pull-up به پایه PB3 وصل است.

در کد برنامه، بررسی این کلید در بالاترین اولویت (اول حلقه اصلی) است. این بخش فقط زمانی فعال می‌شود که زاویه موتور به  $180^\circ$  (گام ۱۰۰) برسد و پایه PB3 صفر شود. با زدن کلید در این زاویه، سیستم وارد یک حلقه while می‌شود. در این حلقه، موتور پله‌پله به عقب برمی‌گردد و مقدار زاویه مدام کم می‌شود. برای بازگشت سریع موتور، زمان تاخیر این حلقه روی  $10$  میلی‌ثانیه تنظیم شد. این روند تا رسیدن به زاویه  $0^\circ$  ادامه دارد و زاویه لحظه‌ای روی نمایشگر چاپ می‌شود.

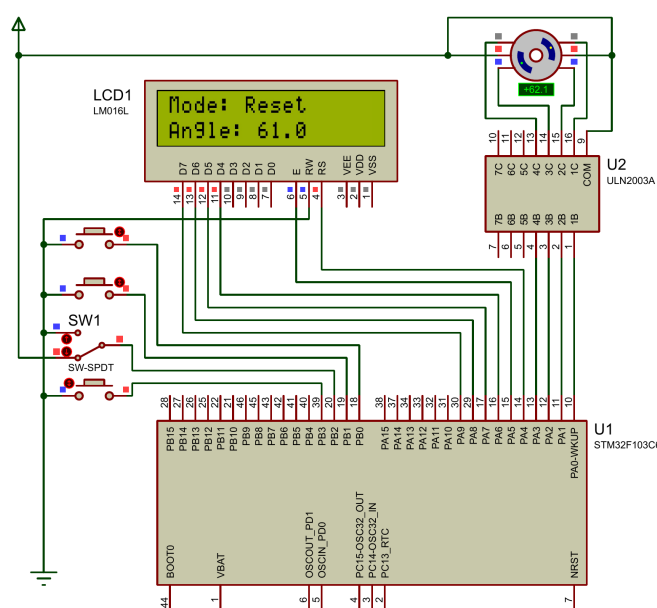
فیلم عملکرد مدار در این حالت ضبط شد و در پوشه Q2 قرار دارد. در تصاویر زیر تنظیمات پایه‌ها، شماتیک نهایی و بازگشت سریع موتور آمده است.



Reset (ب) شماتیک نهایی مدار با اضافه شدن کلید



(آ) تنظیم پایه PB3 به عنوان ورودی در نرم‌افزار



(ج) خروجی مدار در حالت Reset و بازگشت سریع به موقعیت اولیه

شکل ۹: پیاده‌سازی نهایی و تست کلید بازگشت

### ث) مقایسه مدهای تحریک Full-Step و Half-Step

در حالت Full-Step، با هر پالسی که به درایور ارسال می‌شود، روتور به اندازه یک گام کامل (در اینجا  $1/8^\circ$ ) می‌چرخد. اما در حالت Half-Step، توالی تحریک سیم‌پیچ‌ها به شکلی تغییر می‌کند که موتور با هر پالس، تنها نصف یک گام کامل (یعنی  $0/9^\circ$ ) حرکت می‌کند.

اگر شبیه‌سازی را در حالت Half-Step انجام می‌دادیم، دقت سیستم دو برابر می‌شد و حرکت پل خورشیدی بسیار نرم‌تر صورت می‌گرفت. اما از طرف دیگر، برای رسیدن به زاویه  $180^\circ$ ، میکروکنترلر باید به جای  $100$  پالس،  $200$  پالس به درایور ارسال می‌کرد که به معنی دو برابر شدن تعداد مراحل در برنامه است.

#### امکان پیاده‌سازی تغییرات $2^\circ$ و $10^\circ$ با مد Full-Step:

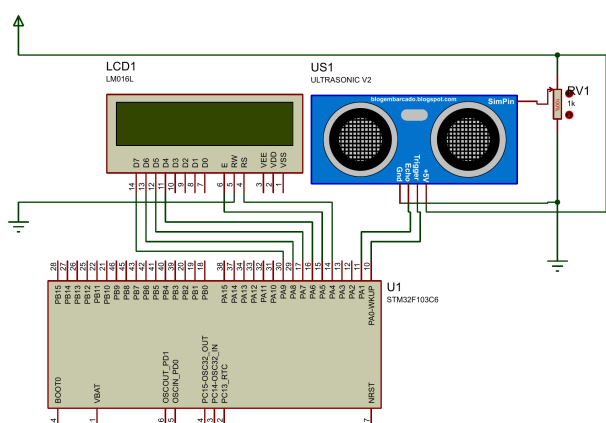
پاسخ به این سوال به شرایط سخت‌افزاری بستگی دارد. اگر بخواهیم از همان استپر موتور قبلی با  $1/8^\circ$  step angle استفاده کنیم، این کار امکان‌پذیر نیست؛ زیرا در حالت Full-Step، موتور تنها می‌تواند مضارب صحیح از  $1/8^\circ$  را طی کند (مثل  $1/8^\circ$ ،  $3/6^\circ$ ،  $5/4^\circ$  و ...) و اعداد  $2^\circ$  و  $10^\circ$  مضرب صحیحی از  $1/8^\circ$  نیستند. اما اگر منظور سوال تغییر فیزیکی موتور (یا تغییر تنظیمات Step Angle در نرم‌افزار پروتئوس) به یک موتور با زاویه گام  $2^\circ$  باشد، به این کار به راحتی در مد Full-Step امکان‌پذیر است. در این حالت با هر پالس، موتور  $2^\circ$  می‌چرخد و در حالت Auto نیز با ارسال  $5$  پالس متوالی، تغییر  $10^\circ$  به طور دقیق به دست می‌آید.

## سوال ۳

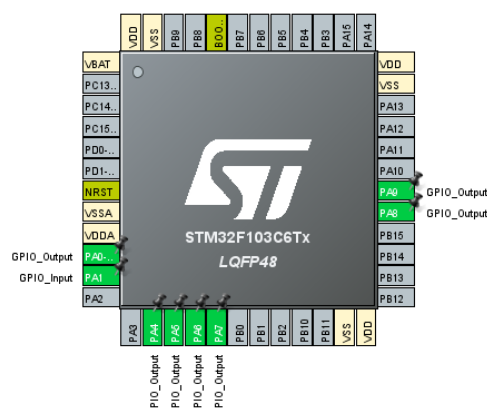
## الف) شبیه‌سازی سنسور آلتراسونیک و اندازه‌گیری فاصله

در این بخش هدف ما محاسبه فاصله با استفاده از سنسور آلتراسونیک و نمایش آن روی LCD است. برای این کار مدار مورد نیاز در محیط پروتئوس بسته شد. پایه تریگر سنسور به پایه  $PA0$  میکروکنترلر وصل شد تا به عنوان خروجی، فرمان شروع را صادر کند. پایه  $PA1$  نیز به پایه میکروکنترلر متصل شد تا به عنوان ورودی، زمان بازگشت موج را دریافت کند. همچنین برای شبیه‌سازی تغییرات فاصله، از یک پتانسیومتر استفاده کردیم که به پایه شبیه‌ساز سنسور وصل شده است. پایه‌های  $PA4$  تا  $PA9$  نیز برای ارتباط با LCD تنظیم شدند.

در نرم‌افزار STM32CubeIDE، پایه‌های مورد نظر به درستی روی حالت ورودی و خروجی تنظیم شدند و برای محاسبه دقیق زمان بازگشت موج، تایمر ۳ را روی دقت ۱ میکروثانیه تنظیم کردیم. تصاویر مربوط به شماتیک مدار و تنظیمات پایه‌ها در نرم‌افزار در ادامه آورده شده است.



(ب) شماتیک مدار سنسور آلتراسونیک در پروتئوس



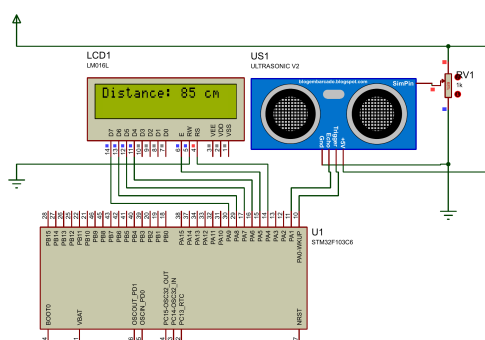
(ت) تنظیمات پایه‌ها در نرم‌افزار

شکل ۱۰: تنظیمات اولیه و سخت‌افزاری برای بخش الف

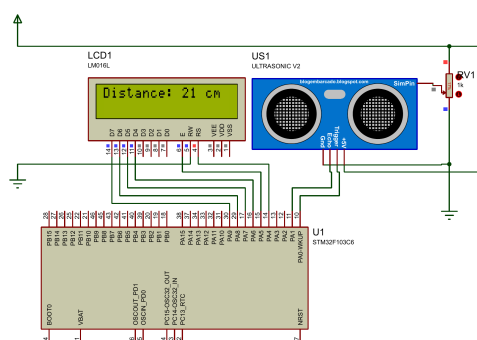
منطق کار به این شکل پیاده‌سازی شد که ابتدا یک پالس با عرض ۱۰ میکروثانیه به پایه تریگر سنسور می‌فرستیم تا سنسور موج صوتی را ارسال کند. سپس میکروکنترلر منتظر می‌ماند تا وضعیت پایه  $PA1$  اکو یک شود. به محض یک شدن این پایه، تایمر شروع به شمارش می‌کند و زمانی که پایه  $PA0$  دوباره صفر شد، مقدار تایمر خوانده می‌شود. این مقدار نشان‌دهنده زمان رفت و برگشت موج صوتی بر حسب میکروثانیه است.

برای به دست آوردن فاصله، این زمان را در سرعت صوت (حدود ۳۴۰ متر بر میکروثانیه) ضرب کرده و بر ۲ تقسیم می‌کنیم (زیرا موج مسیر رفت و برگشت را طی کرده است). در نهایت مقدار محاسبه شده به صورت یک عدد صحیح روی خط اول نمایشگر چاپ می‌شود.

با تغییر دادن مقدار پتانسیومتر در زمان شبیه‌سازی، متوجه می‌شویم که فاصله به درستی محاسبه و به‌روزرسانی می‌شود. در تصاویر زیر خروجی سیستم برای فواصل ۲۱ سانتی‌متر و ۸۵ سانتی‌متر نشان داده شده است. همچنین فیلم عملکرد مدار برای این بخش ضبط شده و در پوشه Q3 قرار دارد.



(ب) محاسبه فاصله ۸۵ سانتی‌متر



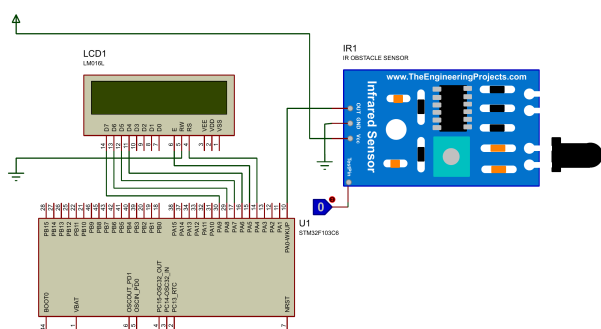
(ت) محاسبه فاصله ۲۱ سانتی‌متر

شکل ۱۱: خروجی مدار در شبیه‌سازی با فواصل مختلف

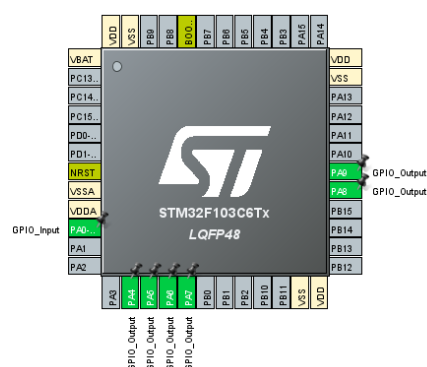
## (ب) شبیه‌سازی سنسور مجاورتی مادون قرمز (IR)

در این بخش هدف ما راه‌اندازی سنسور مجاورتی مادون قرمز و تشخیص حضور جسم است. نحوه کارکرد این سنسور به این صورت است که یک فرستنده به طور پیوسته امواج مادون قرمز ساطع می‌کند. اگر جسمی در مقابل سنسور قرار بگیرد، این امواج به جسم برخورد کرده و به سمت گیرنده بازتاب می‌شوند. در این حالت، خروجی سنسور تغییر وضعیت می‌دهد و میکروکنترلر از حضور جسم مطلع می‌شود.

برای پیاده‌سازی این سیستم در محیط پروتئوس، کتابخانه سنسور مادون قرمز اضافه شد. پایه خروجی سنسور به پایه  $PA0$  میکروکنترلر متصل شد تا به عنوان ورودی، وضعیت سنسور را بخواند. برای شبیه‌سازی حضور یا عدم حضور جسم نیز از یک کلید منطقی استفاده کردیم که به پایه تست سنسور وصل شده است. پایه‌های  $PA4$  تا  $PA9$  نیز مانند بخش‌های قبل برای راه‌اندازی نمایشگر LCD تنظیم شدند.



(ب) شماتیک مدار سنسور مادون قرمز در پروتئوس



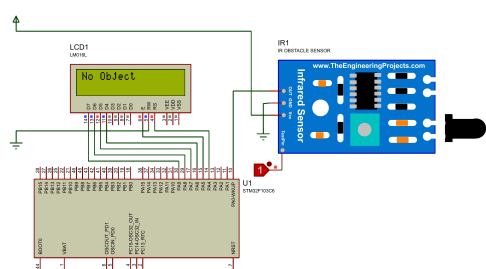
(ت) تنظیمات پایه‌ها در نرم‌افزار

شکل ۱۲: تنظیمات اولیه و سخت‌افزاری برای بخش ب

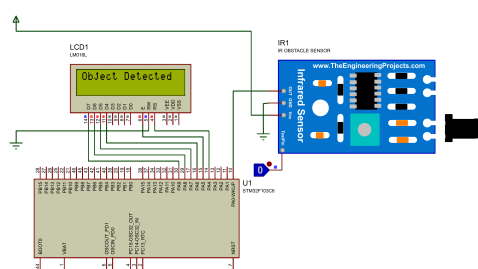
برنامه نوشته شده برای این بخش بسیار ساده است و برعکس بخش قبل، نیازی به استفاده از تایمر ندارد. میکروکنترلر به

طور مداوم، وضعیت پایه  $PA^0$  را بررسی می‌کند. اگر مقدار این پایه صفر شود (به این معنی که سنسور بازتاب امواج را دریافت کرده و خروجی صفر شده است)، عبارت Object Detected روی نمایشگر چاپ می‌شود. در غیر این صورت، میکروکنترلر عبارت No Object را نمایش می‌دهد.

با تغییر دادن وضعیت منطقی پایه تست در شبیه‌سازی بین مقادیر ۰ و ۱، عملکرد مدار مورد بررسی قرار گرفت که سیستم به درستی کار می‌کند. در تصاویر زیر خروجی مدار در هر دو حالت نشان داده شده است. همچنین فیلم عملکرد این سیستم ضبط شده و در پوشه Q3 قرار دارد.



(ب) عدم حضور جسم (وضعیت منطقی یک)



(آ) تشخیص جسم (وضعیت منطقی صفر)

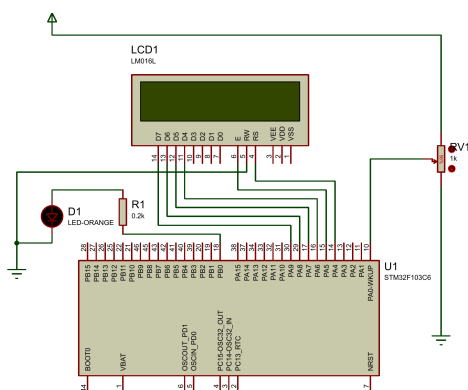
شکل ۱۳: خروجی مدار در شبیه‌سازی برای تشخیص مانع

## سوال ۴

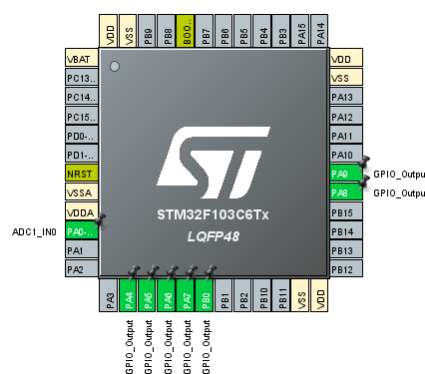
## بخش اول: شبیه‌سازی با استفاده از پتانسیومتر برای اندازه‌گیری زاویه

در این بخش هدف ما خواندن مقدار آنالوگ یک پتانسیومتر، تبدیل آن به زاویه و نمایش آن روی نمایشگر است. همچنین باید در زوایای بیشتر از  $180^\circ$  یک LED را روشن کنیم.

برای پیاده‌سازی سخت‌افزاری در محیط پروتئوس، خروجی پتانسیومتر به پایه  $PA0$  میکروکنترلر متصل شد تا تغییرات ولتاژ به عنوان ورودی آنالوگ دریافت شود. یک LED نیز با استفاده از یک مقاومت به پایه  $PB0$  وصل شد. پایه‌های  $PA4$  تا  $PA9$  هم مانند قبل برای ارتباط با نمایشگر LCD تنظیم شدند. در محیط تنظیمات STM32CubeIDE، پایه  $PA0$  روی حالت  $ADC1\_IN0$  قرار گرفت تا ADC برای این پایه فعال شود. همچنین Continuous Conversion Mode را فعال کردیم تا میکروکنترلر به صورت مداوم تغییرات پتانسیومتر را بخواند.



(ب) شماتیک مدار اندازه‌گیری زاویه در پروتئوس



(ت) تنظیمات پایه‌ها در نرم‌افزار

شکل ۱۴: تنظیمات اولیه و سخت‌افزاری برای بخش اول سوال ۴

**محاسبه زاویه و دقت اندازه‌گیری:** ADC در میکروکنترلر STM32F103 دارای رزولوشن ۱۲ بیتی است. این یعنی ولتاژ ورودی به عددی دیجیتال در بازه ۰ تا  $4095$  ( $2^{12} - 1 = 4095$ ) تبدیل می‌شود. از طرف دیگر، شفت پتانسیومتر در بازه ۰ تا  $270^\circ$  درجه می‌چرخد. برای پیدا کردن زاویه لحظه‌ای فرمول تبدیل به این شکل به دست می‌آید:

$$Angle = \frac{ADC\_Value \times 270}{4095}$$

همچنین برای محاسبه دقت سیستم (Resolution)، بررسی می‌کنیم که هر یک واحد تغییر در خروجی ADC معادل چند درجه

است:

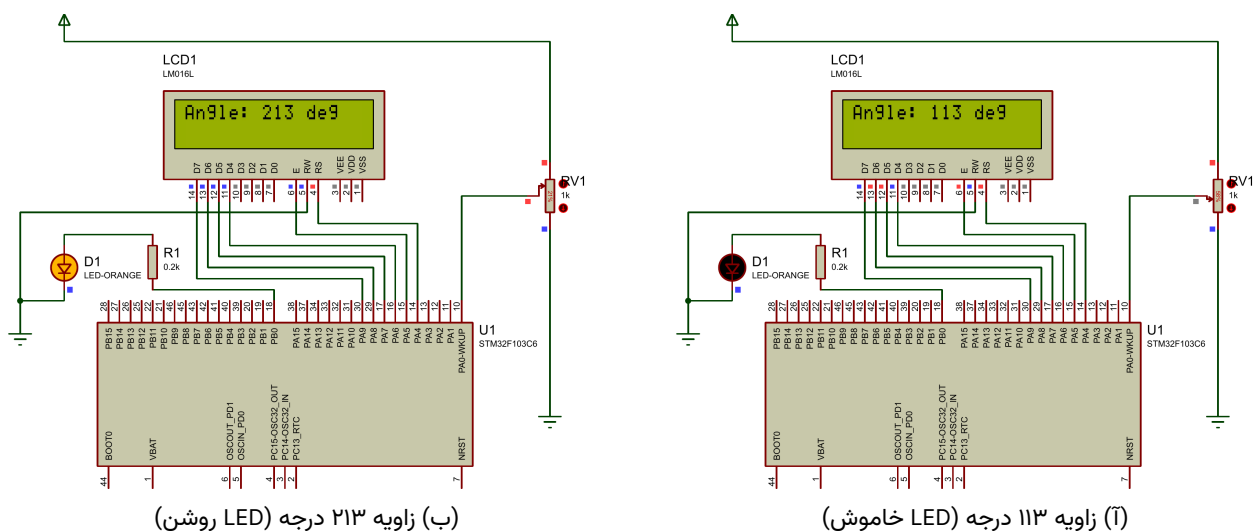
$$Resolution = \frac{270}{4095} \approx 0.0659^\circ$$

به این معنی که سیستم طراحی شده می‌تواند تغییرات زاویه را با دقت حدود  $0.0659^\circ$  درجه اندازه‌گیری کند.

## منطق برنامه‌نویسی و خروجی:

میکروکنترلر به صورت پیوسته مقدار ADC را می‌خواند، آن را در فرمول بالا قرار می‌دهد و زاویه محاسبه شده را روی LCD چاپ می‌کند. سپس با استفاده از یک شرط ساده بررسی می‌شود که اگر زاویه از عدد ۱۸۰ بیشتر بود، پایه  $PB0$  در وضعیت یک قرار بگیرد تا LED روشن شود. در غیر این صورت این پایه صفر باقی می‌ماند.

در شبیه‌سازی پروتئوس با تغییر دادن مقدار پتانسیومتر، عملکرد برنامه بررسی شد. همان‌طور که در تصاویر زیر مشخص است، در زاویه ۱۱۳ درجه LED خاموش است و با عبور از مرز تعیین شده و رسیدن به زاویه ۲۱۳ درجه، LED روشن می‌شود.



شکل ۱۵: خروجی مدار در زوایای مختلف و عملکرد سیستم هشدار

فیلم عملکرد این مدار در پوشه Q4 قرار گرفته است.